occurs in photography. The density (darkness) of a negative is equal to the logarithm of the brightness in the final photograph. This means that any manipulation of the negative during the development stage is a type of homomorphic processing.

Before leaving this example, there is a nuisance that needs to be mentioned. As discussed in Chapter 6, when an N point signal is convolved with an M point filter kernel, the resulting signal is $N+M-1$ points long. Likewise, when an $M \times M$ image is convolved with an $N \times N$ filter kernel, the result is an $(M+N-1) \times (M+N-1)$ image. The problem is, it is often difficult to manage a changing image size. For instance, the allocated memory must change, the video display must be adjusted, the array indexing may need altering, etc. The common way around this is to *ignore* it; if we start with a 512×512 image, we want to end up with a 512×512 image. The pixels that do not fit within the original boundaries are discarded.

While this keeps the image size the same, it doesn't solve the whole problem; these is still the *boundary condition* for convolution. For example, imagine trying to calculate the pixel at the upper-right corner of (d). This is done by centering the Gaussian PSF on the upper-right corner of (c). Each pixel in (c) is then multiplied by the corresponding pixel in the overlaying PSF, and the products are added. The problem is, three-quarters of the PSF lies outside the defined image. The easiest approach is to assign the undefined pixels a value of zero. This is how (d) was created, accounting for the dark band around the perimeter of the image. That is, the brightness smoothly decreases to the pixel value of zero, exterior to the defined image.

Fortunately, this dark region around the boarder can be corrected (although it hasn't been in this example). This is done by dividing each pixel in (d) by a correction factor. The correction factor is the fraction of the PSF that was immersed in the input image when the pixel was calculated. That is, to correct an individual pixel in (d), imagine that the PSF is centered on the corresponding pixel in (c). For example, the upper-right pixel in (c) results from only 25% of the PSF overlapping the input image. Therefore, correct this pixel in (d) by dividing it by a factor of 0.25. This means that the pixels in the center of (d) will not be changed, but the dark pixels around the perimeter will be brightened. To find the correction factors, imagine convolving the filter kernel with an image having all the pixel values equal to *one*. The pixels in the resulting image are the correction factors needed to eliminate the edge effect.

Fourier Image Analysis

Fourier analysis is used in image processing in much the same way as with one-dimensional signals. However, images do not have their information encoded in the frequency domain, making the techniques much less useful. For example, when the Fourier transform is taken of an *audio* signal, the confusing time domain waveform is converted into an easy to understand frequency